

Distributed Computing

Practical Exam – Algorithms & Examples

All 10 Practicals

Practical 1 – RPC: Client-Server Sum with Performance Comparison

Algorithm / Pseudocode

Server Algorithm

```
PROCEDURE RPCServer:
  1. Open ServerSocket on port 5000
  2. LOOP:
    clientSocket = Accept incoming connection
    num1 = Read integer from clientSocket
    num2 = Read integer from clientSocket
    sum = num1 + num2           // Remote Procedure: compute sum
    Send sum back to clientSocket
    Close clientSocket
```

Client Algorithm

```
PROCEDURE RPCClient(num1, num2):
  // -- Local call timing --
  localStart = currentTimeNano()
  localResult = localSum(num1, num2)
  localEnd = currentTimeNano()
  localTime = localEnd - localStart

  // -- RPC call timing --
  rpcStart = currentTimeNano()
  socket = Connect to server (localhost:5000)
  Send num1, num2 over socket
  rpcResult = Read integer from socket
  rpcEnd = currentTimeNano()
  rpcTime = rpcEnd - rpcStart

  Print localResult, localTime
  Print rpcResult, rpcTime
  Print "Overhead = rpcTime - localTime"
```

Example

Input: num1 = 10, num2 = 20

Local Function Call:

```
localSum(10, 20) = 30
Time: ~200 nanoseconds
```

RPC Call:

```
Client sends 10, 20 → Server computes 30 → Client receives 30
Time: ~1,500,000 nanoseconds (network + socket overhead)
```

Performance Comparison:

```
Local call time : 200 ns
RPC call time   : 1,500,000 ns
RPC is ~7500x slower than local call
Overhead of RPC : 1,499,800 ns
```

Key Insight: The local call is near-instant (just an arithmetic op). The RPC call involves TCP connection setup, data serialisation, network round-trip, and deserialisation — hence thousands of times slower.

Practical 2 – RMI: Remote Arithmetic Operations

Algorithm / Pseudocode

Remote Interface

```
INTERFACE ArithmeticInterface extends Remote:
    add(a, b)      throws RemoteException → double
    subtract(a, b) throws RemoteException → double
    multiply(a, b) throws RemoteException → double
    divide(a, b)   throws RemoteException → double
```

Server Algorithm

```
PROCEDURE ArithmeticServer:
    1. Extend UnicastRemoteObject, implement ArithmeticInterface
    2. Implement each method (add, subtract, multiply, divide)
       divide: if b == 0 throw RemoteException("Cannot divide by zero")
    3. createRegistry(1099)
    4. Naming.rebind("rmi://localhost/ArithmeticService", this)
    5. Server now listens for RMI calls
```

Client Algorithm

```
PROCEDURE ArithmeticClient(num1, num2):
    1. stub = Naming.lookup("rmi://localhost/ArithmeticService")
    2. result_add = stub.add(num1, num2)
    3. result_sub = stub.subtract(num1, num2)
    4. result_mul = stub.multiply(num1, num2)
    5. result_div = stub.divide(num1, num2) // catch RemoteException if b=0
    6. Print all results
```

Example

Input: num1 = 15, num2 = 5

```
Addition      : 15 + 5 = 20.0
Subtraction    : 15 - 5 = 10.0
Multiplication : 15 * 5 = 75.0
Division       : 15 / 5 = 3.0
```

Input: num1 = 8, num2 = 0

```
Division       : Error - Cannot divide by zero!
```

Key Insight: The client calls methods on the remote stub exactly as if they were local. The RMI runtime marshals arguments, sends them over the network, and unmarshals the return value transparently.

Practical 3 – Berkeley's Clock Synchronization Algorithm

Algorithm / Pseudocode

```
PROCEDURE BerkeleySync(nodes[0..n-1], clocks[0..n-1]):  
    // Node 0 is the Coordinator (Master)  
  
    STEP 1 - Poll:  
        Coordinator sends TIME_REQUEST to all slave nodes.  
        Each slave replies with its current clock time.  
  
    STEP 2 - Average:  
        total = sum of all clock values (including coordinator's own clock)  
        avgTime = total / n  
  
    STEP 3 - Adjustment:  
        FOR each node i:  
            adjustment[i] = avgTime - clocks[i]  
            // positive → node must advance its clock  
            // negative → node must retard its clock  
  
    STEP 4 - Synchronise:  
        Coordinator sends adjustment[i] to each node i.  
        Each node applies: clock[i] = clock[i] + adjustment[i]  
        All nodes are now synchronised to avgTime.
```

Example

Setup: 4 nodes (Node 0 = Coordinator)

```
Node 0 (Coordinator) clock : 100  
Node 1 clock                : 105  
Node 2 clock                : 90  
Node 3 clock                : 111
```

Step 1 – Poll:

Coordinator collects: [100, 105, 90, 111]

Step 2 – Compute Average:

```
Sum = 100 + 105 + 90 + 111 = 406  
Average = 406 / 4 = 101 (integer division)
```

Step 3 – Compute Adjustments:

```
Node 0: 101 - 100 = +1 (advance by 1)  
Node 1: 101 - 105 = -4 (retard by 4)  
Node 2: 101 - 90 = +11 (advance by 11)  
Node 3: 101 - 111 = -10 (retard by 10)
```

Step 4 – Synchronised Clocks:

```
Node 0: 100 + 1 = 101  
Node 1: 105 - 4 = 101  
Node 2: 90 + 11 = 101  
Node 3: 111 - 10 = 101  
All clocks now = 101
```

Key Insight: Berkeley's algorithm does NOT require a time server. Instead, the coordinator averages all clocks including its own, then sends each node a relative offset — nodes never need to know the 'true' time.

Practical 4 – Bully Algorithm for Leader Election

Algorithm / Pseudocode

```
PROCEDURE BullyElection(initiator, processes[], isAlive[]):

    STEP 1 - Initiator sends ELECTION to higher-ID processes:
    FOR each process p where p.id > initiator.id:
        IF p is alive:
            Send ELECTION message to p
            p sends back OK to initiator
        ELSE:
            No response (crashed)

    STEP 2 - Decision:
    IF no OK received:
        initiator becomes NEW COORDINATOR
        Send COORDINATOR message to all lower-ID processes
    ELSE:
        Higher processes take over election (repeat Step 1 for each)

    STEP 3 - New Coordinator announces:
    The highest alive process sends COORDINATOR to all live processes.

RULE: The process with the HIGHEST ID among alive processes always wins.
```

Example

Setup: 5 processes with IDs [1, 3, 5, 7, 9]. Process 9 (coordinator) crashes. Process 3 detects failure and starts election.

Round 1 – Process 3 sends ELECTION:

```
P3 → P5 : ELECTION | P5 → P3 : OK
P3 → P7 : ELECTION | P7 → P3 : OK
P3 → P9 : ELECTION | No response (crashed)
```

Round 2 – P5 and P7 now run their own elections:

```
P5 → P7 : ELECTION | P7 → P5 : OK
P5 → P9 : ELECTION | No response (crashed)

P7 → P9 : ELECTION | No response (crashed)
P7 receives no OK → P7 declares itself COORDINATOR
```

Announcement:

```
P7 → P1 : COORDINATOR
P7 → P3 : COORDINATOR
P7 → P5 : COORDINATOR
```

```
==> New Coordinator is Process 7 <==
```

Key Insight: The algorithm is called 'Bully' because the higher-ID process always 'bullies' the lower ones into accepting it as coordinator.

Practical 5 – Ring Algorithm for Leader Election

Algorithm / Pseudocode

```
PROCEDURE RingElection(processes[], isAlive[], initiator):

  STEP 1 - Create ELECTION message:
    msg = [initiator.id]
    Send msg to next alive neighbor in ring

  STEP 2 - Forward around ring:
    WHILE message has not returned to initiator:
      current = next alive process in ring
      IF current is alive:
        Append current.id to msg
        Forward msg to current's next alive neighbor
      ELSE:
        Skip (crashed), move to next

  STEP 3 - Identify winner:
    When msg returns to initiator:
      maxId = MAX(all IDs in msg)
      maxId is the NEW COORDINATOR

  STEP 4 - Circulate COORDINATOR message:
    Send COORDINATOR(maxId) around the ring.
    Each alive process acknowledges.
```

Example

Setup: Ring order [2, 4, 6, 8, 10]. Process 8 crashes. Process 2 starts election.

Step 1 – ELECTION message created:

```
P2 creates message [2] → sends to P4
```

Step 2 – Message travels around ring:

```
P4 adds its ID → [2, 4] → sends to P6
P6 adds its ID → [2, 4, 6] → sends to P8
P8 is CRASHED, skipping...
P10 adds its ID → [2, 4, 6, 10] → sends to P2
```

Step 3 – Initiator finds winner:

```
Message [2, 4, 6, 10] returned to P2
MAX = 10
==> Process 10 is the NEW COORDINATOR <==
```

Step 4 – COORDINATOR message circulated:

```
P4 acknowledges: coordinator is 10
P6 acknowledges: coordinator is 10
P10 acknowledges: coordinator is 10
```

Key Insight: Unlike the Bully algorithm, the Ring algorithm uses only one message type and each process adds its ID exactly once. No process sends directly to all others — only to its neighbor.

Practical 6 – Ricart-Agrawala Algorithm (Distributed Mutual Exclusion)

Algorithm / Pseudocode

```
PROCEDURE RicartAgrawala(n processes, each with timestamp):

  When process Pi WANTS to enter CS:
    1. Record timestamp Ti = Lamport_clock(Pi)
    2. Send REQUEST(Ti, i) to ALL other processes
    3. Wait until REPLY received from ALL n-1 processes
    4. Enter CS
    5. Exit CS → send deferred REPLYs

  When process Pj RECEIVES REQUEST(Ti, i) from Pi:
    IF Pj is NOT in CS AND NOT wanting CS:
      Send REPLY to Pi immediately
    ELSE IF Pj IS in CS:
      Defer REPLY to Pi (add to queue)
    ELSE (Pj also wants CS):
      Compare timestamps:
        IF Ti < Tj (Pi has earlier request): Send REPLY to Pi
        IF Ti > Tj (Pj has earlier request): Defer REPLY (add Pi to queue)
        IF Ti == Tj (tie): lower process ID wins → that one gets REPLY
```

Example

Setup: 3 processes. P₀ (ts=2) and P₁ (ts=5) both want CS. P₂ does not want CS.

P₀ sends REQUEST(ts=2, id=0) to P₁ and P₂:

```
P2 receives REQUEST from P0: Not interested in CS → sends REPLY to P0
P1 receives REQUEST from P0: Both want CS, ts(P0)=2 < ts(P1)=5 → sends REPLY to P0
```

P₀ receives 2/2 REPLYs → Enters and Exits CS.

P₁ sends REQUEST(ts=5, id=1) to P₀ and P₂:

```
P2 receives REQUEST from P1: Not interested → sends REPLY to P1
P0 receives REQUEST from P1: P0 now exited CS → sends deferred REPLY to P1
```

P₁ receives 2/2 REPLYs → Enters and Exits CS.

Execution Order:

1. Process 0 (timestamp = 2) ← lower timestamp, goes first
2. Process 1 (timestamp = 5)

Key Insight: No central coordinator is needed. A process can enter CS only after collecting permission from every other process. Timestamps ensure a consistent, deadlock-free order.

Practical 7 – Raymond's Tree-Based Algorithm (Distributed Mutual Exclusion)

Algorithm / Pseudocode

```
SETUP:
  Processes arranged as a logical tree.
  A single TOKEN exists in the system.
  Each process maintains: holder → neighbor in direction of token
  Token holder points to itself.

PROCEDURE Raymond_RequestCS(requester):
  IF requester holds TOKEN:
    Enter CS directly
  ELSE:
    1. Send REQUEST to holder[requester]
    2. Each intermediate process i:
       Add requester to local queue
       IF i has token AND queue not empty:
         Send TOKEN to head of queue
       ELSE:
         Forward REQUEST to holder[i]
    3. Token travels back along the path to requester
    4. Requester receives TOKEN → Enter CS
    5. On exit, update holder pointers:
       All nodes on the path now point toward the new token holder
```

Example

Setup: 5-node tree. P0 is root; P1, P2 are children of P0; P3, P4 are children of P1. P2 currently holds the TOKEN.

Initial Holder Pointers:

```
P0: holder → P2 (token is down through P0 toward P2)
P1: holder → P0 (token is up through P0)
P2: holder → self (TOKEN HOLDER)
P3: holder → P1
P4: holder → P1
```

P3 wants to enter CS:

```
Step 1 - REQUEST propagation:
P3 → P1 : REQUEST
P1 → P0 : REQUEST
P0 → P2 : REQUEST

Step 2 - TOKEN transfer (P2 is idle / done with CS):
P2 → P0 : TOKEN
P0 → P1 : TOKEN
P1 → P3 : TOKEN
```

P3 receives TOKEN → Enters CS → Exits CS.

Updated Holder Pointers:

```
P2: holder → P0 (token moved away)
P0: holder → P1
P1: holder → P3
P3: holder → self (NEW TOKEN HOLDER)
```

Key Insight: Raymond's algorithm reduces message complexity to $O(\log n)$ for a balanced tree. Requests travel up toward the token, and the token travels back down to the requester.

Practical 8 – Chandy-Misra-Haas Algorithm (Distributed Deadlock Detection)

Algorithm / Pseudocode

```
SETUP:
  Wait-For Graph: edge  $P_i \rightarrow P_j$  means  $P_i$  is blocked waiting for  $P_j$ 
  A cycle in the WFG = DEADLOCK

PROCEDURE CMH_DetectDeadlock(initiator  $P_i$ ):
  1. IF  $P_i$  is NOT waiting  $\rightarrow$  No deadlock to detect.
  2.  $P_i$  sends PROBE( $i, i, \text{waitFor}[i]$ )
     // PROBE(initiator, sender, receiver)

  3. When PROBE( $i, j, k$ ) arrives at  $P_k$ :
     IF  $k == i$ :
       DEADLOCK DETECTED (probe returned to initiator)
       Report cycle and stop.
     ELSE IF  $P_k$  is blocked ( $\text{waitFor}[k] \neq -1$ ):
       Forward PROBE( $i, k, \text{waitFor}[k]$ ) to  $\text{waitFor}[k]$ 
     ELSE:
        $P_k$  is NOT blocked  $\rightarrow$  Discard probe (no deadlock from this path)
```

Example

Setup: 4 processes with the following wait-for relationships:

```
P0 waits for P1
P1 waits for P2
P2 waits for P3
P3 waits for P0    ← cycle!
```

Deadlock Detection initiated by P0:

```
P0 sends PROBE(0, 0, 1)

Step 1: PROBE(0, 0, 1) arrives at P1
  1 != 0 (initiator), P1 is blocked (waiting for P2)
  P1 forwards PROBE(0, 1, 2)

Step 2: PROBE(0, 1, 2) arrives at P2
  2 != 0, P2 is blocked (waiting for P3)
  P2 forwards PROBE(0, 2, 3)

Step 3: PROBE(0, 2, 3) arrives at P3
  3 != 0, P3 is blocked (waiting for P0)
  P3 forwards PROBE(0, 3, 0)

Step 4: PROBE(0, 3, 0) arrives at P0
  receiver (0) == initiator (0)  $\rightarrow$  DEADLOCK DETECTED!

Deadlock cycle:  $P_0 \rightarrow P_1 \rightarrow P_2 \rightarrow P_3 \rightarrow P_0$ 
```

Key Insight: No global knowledge is needed. The probe travels along the WFG edges; if it returns to its origin, a cycle (deadlock) exists.

Practical 9 – Dynamic Load Balancing: Least Connections Method

Algorithm / Pseudocode

```
SETUP:
    servers[]      = list of servers
    connections[]  = current active connections per server

PROCEDURE LeastConnections(newRequest):
    1. minServer = servers[0]
       FOR each server s in servers[1..]:
           IF connections[s] < connections[minServer]:
               minServer = s

    2. Assign newRequest to minServer
       connections[minServer] += 1

    3. When a task completes on server s:
       connections[s] -= 1

REPEAT for every incoming request.
// Dynamic: decision updates in real-time as connections change.
```

Example

Setup: 3 servers with initial connections:

```
Server 1: 3 connections
Server 2: 1 connection
Server 3: 5 connections
```

Request 1 arrives:

```
Least loaded = Server 2 (1 connection)
Assign Request 1 → Server 2
Server 2: 1 → 2 connections
```

Request 2 arrives:

```
Current: S1=3, S2=2, S3=5
Least loaded = Server 2 (2 connections)
Assign Request 2 → Server 2
Server 2: 2 → 3 connections
```

Task completes on Server 1:

```
Server 1: 3 → 2 connections
```

Request 3 arrives:

```
Current: S1=2, S2=3, S3=5
Least loaded = Server 1 (2 connections)
Assign Request 3 → Server 1
Server 1: 2 → 3 connections
```

Final State:

```
Server 1: 3 connections
Server 2: 3 connections
Server 3: 5 connections
```

Key Insight: Unlike Round Robin, Least Connections accounts for varying task durations. A slow server accumulates more connections and receives fewer new requests — balancing load dynamically.

Practical 10 – Static Load Balancing: Weighted Round Robin Method

Algorithm / Pseudocode

```
SETUP:
  servers[] = list of servers, each with a fixed weight w[i]
  Higher weight → more tasks assigned per round

PROCEDURE WeightedRoundRobin(tasks[], servers[]):
  serverIndex = 0
  FOR each task T in tasks:
    currentCount = 0
    WHILE currentCount < weight[serverIndex] AND tasks remain:
      Assign next task → servers[serverIndex]
      currentCount += 1
    serverIndex = (serverIndex + 1) % numServers

// Static: weights are fixed at configuration time and do not change.
```

Example

Setup: 3 servers with weights, 10 tasks to distribute:

```
Server 1: weight = 3
Server 2: weight = 1
Server 3: weight = 2
Total weight per round = 6 (= 3+1+2)
```

Task Distribution:

```
Round 1:
  Task 1 → Server 1 (slot 1 of 3)
  Task 2 → Server 1 (slot 2 of 3)
  Task 3 → Server 1 (slot 3 of 3)
  Task 4 → Server 2 (slot 1 of 1)
  Task 5 → Server 3 (slot 1 of 2)
  Task 6 → Server 3 (slot 2 of 2)

Round 2:
  Task 7 → Server 1 (slot 1 of 3)
  Task 8 → Server 1 (slot 2 of 3)
  Task 9 → Server 1 (slot 3 of 3)
  Task 10 → Server 2 (slot 1 of 1)
```

Distribution Summary:

```
Server 1: weight=3, tasks received = 6
Server 2: weight=1, tasks received = 2
Server 3: weight=2, tasks received = 2
Total tasks distributed = 10
```

Key Insight: Tasks are distributed proportional to weights (3:1:2). Server 1 gets 3× as many tasks as Server 2 — useful when servers have different CPU/memory capacities.